Student Accommodation Platform

RentEasy

Project Document for the Student Accommodation Web Application

Prepared for development, presentation, and implementation review.

Project	RentEasy
Application Type	Full-stack web application
Primary Users	Students, property owners, administrators
Frontend	React, Vite, Tailwind CSS
Backend	Node.js, Express.js, TypeScript
Database	Supabase Postgres

Generated 2026-07-12

1. Executive Summary

RentEasy is a student accommodation platform for discovering, comparing, and booking PGs, hostels, and mess facilities. Students can browse listings, request bookings, manage profiles, and submit reviews after an eligible completed stay. Owners can create listings, upload multiple room images, maintain vacant-room counts, and manage booking requests. Administrators can monitor users, bookings, and reviews.

The current implementation is a modular monolith: a React frontend communicates with an Express REST API, and the API reads and writes data through the Supabase JavaScript client. The project intentionally focuses on core application behavior instead of advanced infrastructure such as payment gateways, Kafka, Kubernetes, Terraform, or Elasticsearch.

2. Goals and Scope

- Provide a student-focused accommodation discovery experience.
- Let owners manage listings, room availability, multiple photos, and booking requests.
- Support secure login with JWT and role-based access control.
- Use Supabase as the managed database layer.
- Keep the application simple enough for local development and demo presentation.

Out of scope for this version

- Online payment collection and wallet flows.
- Native mobile apps.
- Microservices, event streaming, and search infrastructure.
- Production-grade analytics and automated background jobs.

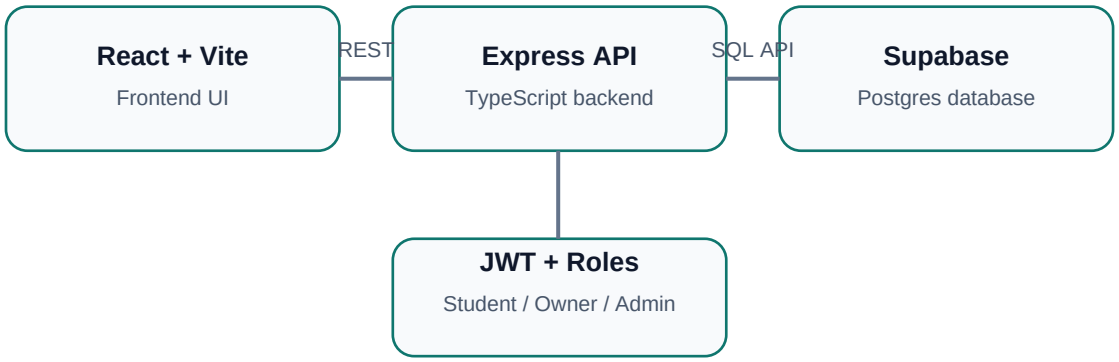
3. User Roles and Capabilities

Role	Main Capabilities
Student	Register, log in, search listings, filter rooms, request bookings, cancel pending bookings, view history, update profile, submit eligible reviews.
Owner	Register, log in, create listings, add multiple images, update vacant rooms, approve/reject bookings, mark stays completed, remove listings.
Admin	View users, bookings, and reviews; update user roles; remove inappropriate reviews.

4. Technology Stack

Layer	Technology
Frontend	React 18, Vite 6, React Router, Axios, Tailwind CSS, lucide-react icons
Backend	Node.js, Express 4, TypeScript, Zod validation, Helmet, CORS, rate limiting
Authentication	JWT token generation, bcrypt password hashing, role-based authorization middleware
Database	Supabase Postgres with service-role backend client
Development	npm scripts, TypeScript builds, Supabase Studio, Git

5. High-Level Architecture



The frontend is responsible for page routing, dashboards, forms, listing cards, notifications, and user interaction. The backend owns authentication, validation, role checks, and database access. Supabase stores the application data in relational tables.

6. Frontend Structure

Area	Files / Purpose
Shell and navigation	AppShell.tsx - dashboard sidebar, header, notification dropdown, logout.
Auth	AuthPage.tsx - login and student/owner registration.
Browse	BrowsePage.tsx - search, filters, apply filters button, room card grid.
Room cards	ListingCard.tsx - compact card, image crop, vacant-room label, multi-photo count.
Listing detail	ListingDetailPage.tsx - image gallery, booking form, review eligibility UI.
Dashboards	DashboardPage.tsx, OwnerDashboardPage.tsx, AdminDashboardPage.tsx.
API layer	services/api.ts - Axios client and typed REST helpers.

7. Backend Modules

Module	Responsibility
auth	Register/login, bcrypt password hashing, JWT issue.
users	Current profile, profile updates, admin user role management.
listings	Public listing search, listing details, owner/admin create/update/delete.
owners	Owner-scoped listing and booking views.
bookings	Booking requests, cancel, approve, reject, complete, availability handling.
reviews	Review creation and listing review reads with duplicate prevention.
admin	Admin booking and review moderation endpoints.

8. REST API Summary

Endpoint Group	Examples
Authentication	POST /api/auth/register, POST /api/auth/login
Users	GET /api/users/me, PATCH /api/users/me, PATCH /api/users/:id/role
Listings	GET /api/listings, GET /api/listings/:id, POST /api/listings, PATCH /api/listings/:id
Bookings	POST /api/bookings, GET /api/bookings/me, PATCH approve/reject/cancel/complete
Owners	GET /api/owners/me/listings, GET /api/owners/me/bookings
Reviews	GET /api/listings/:listingId/reviews, POST /api/listings/:listingId/reviews
Admin	GET /api/admin/bookings, GET /api/admin/reviews, DELETE /api/admin/reviews/:id

9. Database Design

The app uses Supabase Postgres tables in the public schema. The current schema file also includes ALTER TABLE statements for applying newer fields to an existing database.

Table	Important Columns	Purpose
users	id, name, email, password, role, created_at	Login identity and role source.
profiles	id, full_name, email, phone, role, avatar_url	Editable profile information.
listings	id, owner_id, title, type, city, address, price, amenities, photos, available, vacant_rooms	PG, hostel, and mess listings.
bookings	id, listing_id, student_id, status, message, completed_at, created_at	Booking lifecycle and review timing.
reviews	id, listing_id, student_id, rating, comment, created_at	Student ratings and comments.

Important migration SQL

For existing Supabase projects, run the schema migration for listings.vacant_rooms and bookings.completed_at before relying on exact room counts or one-month review timing.

Column	Reason
listings.vacant_rooms	Stores owner-managed number of currently vacant rooms.
bookings.completed_at	Stores when a stay was completed so review eligibility can unlock after one month.

10. Key Workflows

Workflow	Steps
Student booking	Browse listings -> filter/search -> open listing -> send booking request -> track status in dashboard.
Owner listing	Create listing -> add rent, city, amenities, photo URLs, vacant rooms -> manage listing cards.
Booking approval	Owner sees request -> approve or reject -> approval reduces vacancy when vacancy column exists.
Review	Owner marks approved booking completed -> one calendar month passes -> student can submit rating and comment.
Notifications	Header bell loads role-aware booking activity and navigates to the relevant dashboard.

11. Security and Validation

- Passwords are hashed using bcrypt before storage.
- The backend signs JWTs and uses authentication middleware for protected endpoints.
- Role authorization protects student, owner, and admin actions.
- Zod validates request bodies and query parameters.
- Helmet, CORS configuration, and rate limiting are enabled in the backend.
- Supabase service-role credentials must stay only in backend environment files and never be exposed in frontend code.

12. Environment Setup

Area	Required Values
Backend .env	PORT, NODE_ENV, SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, JWT_SECRET, JWT_EXPIRES_IN, FRONTEND_URLS
Frontend .env	VITE_API_URL pointing to the backend API, for example http://localhost:3000/api
Database	Run supabase/schema.sql in Supabase SQL Editor for tables, indexes, and migrations.

Development commands

Location	Command	Purpose
backend	npm run dev	Run API server with tsx watch.
backend	npm run build	Type-check and compile backend.
frontend	npm run dev	Run Vite dev server.
frontend	npm run build	Type-check and build frontend assets.

13. Demo Data

A demo seeding script was used during development to create sample owners, students, listings, bookings, and reviews. For a repeatable production workflow, move this seeding logic into a tracked script such as backend/scripts/seed-demo-data.ts.

Demo Login	Role	Password
ananya.owner@renteasy.test	Owner	Password@123
rahul.owner@renteasy.test	Owner	Password@123
priya.student@renteasy.test	Student	Password@123
aarav.student@renteasy.test	Student	Password@123

14. Current Feature Highlights

- Dashboard-style UI with collapsible sidebar navigation.
- Student profile edit page with booking history tab.
- Owner dashboard for listing creation, booking decisions, and room vacancy updates.
- Compact room cards with better image crop and multi-photo indicator.
- Listing detail gallery with thumbnail switching.
- Filter panel with an always-visible Apply filters button.
- Notification dropdown in the dashboard header.
- Backward-compatible backend fallbacks for databases missing the latest vacancy/completion columns.

15. Known Implementation Notes

- The rooms feature is currently represented by listings.vacant_rooms rather than a separate rooms table.
- Exact vacant-room persistence requires the listings.vacant_rooms column to exist in Supabase.
- One-month review timing requires bookings.completed_at to exist in Supabase.
- Profile fields such as city, college, and programme are frontend-only unless the profiles table is extended.
- The frontend uses external room image URLs; production could add Supabase Storage uploads.

16. Suggested Next Enhancements

Priority	Enhancement
High	Add Supabase Storage uploads for owner room images.
High	Move demo seeding into a committed backend script.
Medium	Add a dedicated rooms table if room-level pricing and occupancy are needed.
Medium	Add automated tests for listing filters, booking lifecycle, and review eligibility.
Low	Add email notifications and owner analytics.

End of document.